operations when *k* gets large, because of the forward propagation. For small numbers of processing units, the first approach will therefore finish faster, whereas for massively parallel systems, the second will finish faster. This is an extreme example of the issues discussed in section 8.2.1.

Anyway, efficiency issues aside, let's look at some code. The following listing shows the first approach.

Listing 8.11. Calculating partial sums in parallel by dividing the problem

```
template<typename Iterator>
void parallel_partial_sum(Iterator first,Iterator last)
{
    typedef typename Iterator::value_type value_type;

    struct process_chunk                                    <-(1)
    {
        void operator()(Iterator begin,Iterator last,
                        std::future<value_type>* previous_end_value,
                        std::promise<value_type>* end_value)
        {
            try
            {
                Iterator end=last;
                ++end;
                std::partial_sum(begin,end,begin);          <-(2)
                if(previous_end_value)                      <-(3)
                {
                    value_type& addend=previous_end_value->get();   <-(4)
                    *last+=addend;                          <-(5)
                    if(end_value)
                    {
```